monolithic structure. Everything was compiled into the kernel, and loaded into memory along with the kernel. Somewhere down the line, the kernel developers realised it would be better if some of the not-so-important stuff could be placed outside the kernel, and only loaded in memory when required. So along came modules, stuff that can be loaded by the user (or automated user level programs), as and when required. All the modules can be found in the ***/lib/modules/kernelversion/*** directory.

In this workshop, we will install the latest Linux kernel (2.6.0-test11) side-by-side your existing kernel (a stable 2.4.xx version).

STEP 1 Before installing the new kernel, you will have to check for the version numbers of a few software installed on your existing system.

First make sure that your gcc compiler version is 2.95.3 or higher. You can check this by using the ***gcc --version*** command.

Next, check the versions of gtk+, glib and libglade, using a combination of the rpm and grep commands:

rpm -qa | grep gtk+
rpm -qa | grep glib
rpm -qa | grep libglade

Look for output lines that say something like:

gtk+-1.2.10-11
glib-1.2.10-5
libglade-0.16-4, respectively.

Before going on to compile and install the 2.6.0-test11 kernel, make sure you have a distribution with a stable and working 2.4.xx release kernel. Upgrading from an older 2.2.XX series kernel to the 2.6.0 kernel is nearly impossible; you might as well install a new distribution rather than upgrade a few hundred packages.

STEP 2 Once you have checked all the prerequisites, you can go ahead and compile the new kernel. Download the latest 2.6.0-test11 kernel from ***www.kernel.org***. Uncompress and untar this package into the ***/usr/src/*** directory. This is the directory where all kernel source code resides.

STEP 3 Next, ***cd*** to ***/usr/src/linux-2.6.0-test11/*** and type in the following command—***make mrproper***.

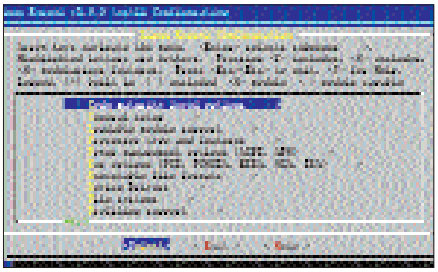
This command is not required during a first time install, but it is recommended to run it each time you recompile, since it removes any intermediate files generated during previous compilations.

STEP 4 The next step is to configure the new kernel to enable all the required options that you want included in the kernel. These configuration parameters are stored in a .config file in the kernel source directory, and can be switched on or off by directly editing the file. However, the contents of the file are cryptic, and various front-ends are available to configure the kernel. The simplest way is to type in ***'make xconfig'***.

This will bring up a dialogue box that can be used to configure the parameters required for compiling the kernel.

In the new 2.6 kernels, there are multiple ways of configuring the kernel. Apart from the older ***make config***, ***make menuconfig*** and ***make xconfig*** commands, there is a ***make gconfig*** command that brings up a gnome-based interface. Also to be noted is that now, ***make xconfig*** does not use a simple tk based interface; it is kde-based, and won't run unless the latest kde libraries are installed. The gnome-based interface also requires the latest gnome libraries, failing which it will not run. While this might be a source of irritation to die-hard Linux fans, it reflects a conscious decision on the part of the kernel developers, who came up with an interface that is easier to use for novice users.

To use ***make gconfig*** you will need gtk+ 2.0, glib-2.0 and libglade-2.0. If these are not installed on your system, it would be better to switch to a newer distribution, as upgrading these packages will be quite tedious. In any case, if you still need to upgrade, you could try the ***make menuconfig*** command that brings up a barebones interface on the console. If you've used a text-based install of Red Hat, you won't have any problems using this configuration utility.



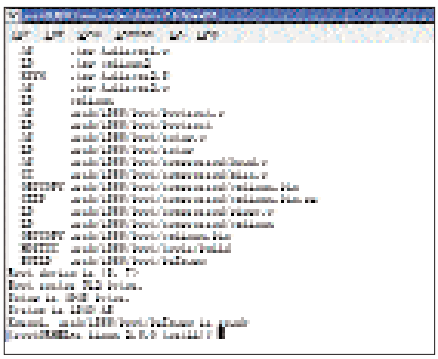
make menuconfig *brings up a barebones interface that can be used to configure kernel parameters before compilation*

STEP 5 Next, use the ***make dep*** command to build all the required dependencies:

After this comes the main step—the creation of the new kernel image. For this, type in ***make bzImage***

This creates a compressed image of the Linux kernel that is used for system booting. The compression is done using the bzip algorithm. In fact, 'bzImage' stands for 'big compressed image', and you can create a different version of the compressed image that is guaranteed to be smaller than 512 KB. If this step goes well, you will get a message saying that the image is ready.

STEP 6 The next step is to use the ***make modules*** command. This command is responsible for compiling all the modules that were configured at the start, and are now in the .config file. After compilation, the modules have to be installed in the



The output of the ***make bzImage*** *command gives the location of the compiled image, and some more detailed information*

Patching the Linux Kernel

Patching the kernel is another approach to keep your kernel up-to-date. In this method, incremental patches are applied to the kernel, the advantage being that you don't have to download the complete kernel source code, just the updated patches. The downside is that you'll have to apply the patches regularly. These patches are available at www.kernel.org. Once downloaded, enter the top level directory of the kernel source (for example, linux-2.6.xx) and then patch it using the following command:

gzip -cd ../patch-2.6.xx.gz | patch -p1
or ***bzip2 -dc ../patch-2.6.xx.bz2 | patch -p1***

Alternatively, you could use the 'patch-kernel' script found in the scripts/ directory in any kernel source directory. The patch-kernel script must be passed a parameter that shows the location of the kernel source tree (for example, /usr/src/linux-2.4.0). It picks up the patch file from the current working directory.